

Сопоставление alg.pdf и реальной системы

Документ подготовлен по результатам сверки файла `research/alg.pdf` с фактической реализацией проекта TechTracker. Цель документа — в читаемом виде показать, какие формулы уже совпадают с кодом, какие требуют уточнения и как их корректно переписать в диссертации.

1. Сопоставление формул и моделей

1.1. Объект исследования

В `alg.pdf` сервер задаётся как вектор наблюдаемых эксплуатационных метрик. Эта часть концептуально совпадает с кодом.

$$X(t) = (x_1(t), x_2(t), \dots, x_n(t)) \in \mathbb{R}^n$$

В текущей реализации вектор `X(t)` включает метрики загрузки CPU, использования памяти, сетевых задержек и трафика, температурные показатели, а также метрики состояния RAID и SSD.

Что переписать в диссертации: оставить формулу без изменений, но явно указать, что `X(t)` объединяет вычислительные, сетевые, температурные и дисковые каналы телеметрии.

1.2. Пространство состояний

В `alg.pdf` скрытое состояние задаётся множеством состояний и вероятностным вектором `\pi_t`. В коде эта часть уже формализована строже: состояние вычисляется не эвристикой, а через признаки и softmax.

$$S = \{S_0, S_1, S_2\}$$

$$\pi_t = (P(S_0 | X_t), P(S_1 | X_t), P(S_2 | X_t))$$

В коде используется вектор признаков состояния:

$$\varphi_t = [r_{agg}, r_{top1}, r_{top3}, r_{temp}, r_{wear}, r_{lat}, p_{Markov, S_2}]^T$$

Для каждого состояния рассчитывается логит:

$$z_k = b_k + \sum_j w_{kj} g_j(\varphi)$$

После этого вероятности нормируются через softmax:

$$P(S_k | X_t) = \frac{\exp(z_k/\tau)}{\sum_l \exp(z_l/\tau)}$$

Что переписать в диссертации: заменить словесное описание скрытого состояния на явную вероятностную модель по признакам.

1.3. STL-декомпозиция

В `alg.pdf` STL описана корректно и совпадает с кодом.

$$x_i(t) = T_i(t) + S_i(t) + \varepsilon_i(t)$$

В реализации после STL в прогнозирование подаётся очищенный ряд без сезонной компоненты:

$$\tilde{x}_i(t) = T_i(t) + \varepsilon_i(t)$$

Что переписать в диссертации: явно указать, что SARIMA получает очищенный ряд $\tilde{x}_i(t)$, а не исходный ряд целиком.

1.4. SARIMA

В `alg.pdf` приведена общая формула SARIMA класса Box–Jenkins. Концептуально она совпадает с кодом, однако практическая реализация сейчас упрощена.

$$\Phi_p(B) \Phi_p(B^S) (1 - B)^d (1 - B^S)^D x_t = \Theta_q(B) \Theta_q(B^S) \varepsilon_t$$

В системе используется класс `SARIMAX`, но не полный перебор всех порядков, а ограниченный набор устойчивых кандидатов.

Что переписать в диссертации: писать, что реализован baseline-модуль на основе SARIMA/SARIMAX с выбором из нескольких устойчивых спецификаций. Не утверждать, что уже реализован полный автоматический Box–Jenkins search.

1.5. LSTM

В `alg.pdf` LSTM подана как многомерная модель по вектору признаков диска. В коде реализован более простой, но рабочий вариант: отдельная одномерная LSTM для каждой метрики.

$$(h_t, c_t) = LSTM(x_t, h_{t-1}, c_{t-1})$$

$$y_{t+1} = W_o h_t + b_o$$

Что переписать в диссертации: нельзя писать, что уже реализована многомерная LSTM по объединённому вектору `Latency + IOPS + TBW + SMART`. Корректная формулировка: в текущей системе используется набор независимых одномерных LSTM-моделей по отдельным метрикам.

1.6. Интеграция прогнозов SARIMA и LSTM

Идея в `alg.pdf` и в коде совпадает, однако в реализации вес назначается отдельно для каждой метрики, а не единым глобальным коэффициентом.

$$y_m(t+h) = \alpha_m y_m^{SARIMA}(t+h) + (1 - \alpha_m) y_m^{LSTM}(t+h)$$

Здесь α_m подбирается автоматически по историческим ошибкам моделей либо по ширине их интервалов, если истории мало.

Что переписать в диссертации: использовать формулу с α_m , то есть с индивидуальным весом для каждой метрики.

1.7. Неопределённость ансамбля

В `alg.pdf` итоговый интервал ансамбля явно не расписан. В текущей системе `uncertainty` учтена отдельной формулой.

$$u_m(h) = \sqrt{(\alpha_m u_m^{SARIMA}(h))^2 + ((1 - \alpha_m) u_m^{LSTM}(h))^2 + (\beta |y_m^{SARIMA} - y_m^{LSTM}|)^2}$$

Тогда итоговый интервал ансамбля задаётся как:

$$p10 = y_m(t + h) - u_m(h), \quad p50 = y_m(t + h), \quad p90 = y_m(t + h) + u_m(h)$$

Что переписать в диссертации: добавить эту формулу как авторское расширение ансамбля для учёта неопределённости и расхождения между SARIMA и LSTM.

1.8. Модель Вейбулла

Базовая формула из `alg.pdf` совпадает с кодом, но в реализации используется более прикладная величина — условная вероятность отказа на горизонте прогноза.

$$S(t) = \exp\left(-\left(\frac{t}{\eta}\right)^\beta\right)$$

$$P_{fail}(h | t_0) = 1 - \frac{S(t_f)}{S(t_0)}$$

Что переписать в диссертации: раздел лучше строить не только вокруг функции выживания $S(t)$, а вокруг горизонтной вероятности отказа $P_{fail}(h | t_0)$.

1.9. Пороговый риск для неизнашиваемых метрик

Этой ветви в `alg.pdf` почти нет, но в реальной системе она необходима и уже реализована.

$$r_m(h) = \text{clip}\left(\frac{p90_m(h)/\theta_m - 0.75}{0.75}, 0, 1\right)$$

Что переписать в диссертации: явно добавить вторую ветвь оценки риска — пороговый риск деградации для CPU, RAM, температуры и сетевых метрик.

1.10. Марковская динамика состояний

В `alg.pdf` формула задана корректно и совпадает с кодом.

$$\pi_{t+k} = \pi_t P^k$$

В реализации матрица переходов оценивается по истории состояний конкретного устройства и дополняется статистикой по серверам того же класса.

Что переписать в диссертации: оставить основную формулу без изменений, но расширить описание процедуры оценки матрицы переходов и сглаживания вероятностей.

1.11. Общий риск отказа

В `alg.pdf` явная формула итогового риска отсутствует. В системе она уже есть.

$$R_{metric}(h) = 0.55 r_w(h) + 0.45 \max_m r_m(h)$$

$$R_{overall}(h) = w_M \pi_{t+k}(S_2) + (1 - w_M) R_{metric}(h)$$

Что переписать в диссертации: добавить эти формулы как авторскую модификацию интегральной модели риска.

1.12. Байесовский риск и АНР

В этой части `alg.pdf` и код хорошо совпадают.

$$R(a_i) = \sum_j P(S_j | X_t) L(a_i, S_j)$$

$$U(a_i) = \sum_k w_k u_k(a_i)$$

Что переписать в диссертации: оставить эти формулы, но дополнительно отметить, что в системе итоговый score решения может быть чисто байесовским либо смешанным: байесовская полезность плюс АНР-полезность.

1.13. Интегральная модель системы

Старая композиция из `alg.pdf` концептуально верна, но в текущей реализации её нужно уточнить.

$$F_{old} = AHP \circ Decision \circ Markov \circ Weibull \circ (\alpha \cdot SARIMA + (1 - \alpha) \cdot LSTM) \circ STL$$

Фактическая композиция системы теперь выглядит точнее так:

$$F = AHP \circ Decision \circ RiskFusion \circ Markov \circ StateInference \circ Ensemble \circ \{SARIMA, LSTM\}$$

Что переписать в диссертации: заменить старую композицию на новую, где есть отдельный модуль оценки скрытого состояния и отдельный модуль объединения рисков.

2. Текст для главы 1

2.1. Формализация объекта исследования

Объектом исследования является серверное оборудование, функционирующее как стохастическая динамическая система, состояние которой изменяется под воздействием эксплуатационной нагрузки, фоновых процессов, деградации аппаратных компонентов и внешних сетевых факторов. В каждый момент времени сервер описывается вектором наблюдаемых телеметрических признаков

$$X(t) = (x_1(t), x_2(t), \dots, x_n(t)) \in R^n$$

где компоненты вектора соответствуют наиболее информативным каналам мониторинга: загрузке процессора, использованию оперативной памяти, сетевым задержкам и объёму трафика, температурным характеристикам, а также метрикам состояния дисковой подсистемы и RAID-массива.

Такое представление позволяет рассматривать сервер как многомерный временной объект, для которого решение о техническом состоянии и риске отказа должно приниматься не по одной метрике, а по их совокупности. При этом измеряемые величины являются наблюдаемыми, тогда как фактическое техническое состояние сервера напрямую не наблюдается и должно быть оценено косвенно.

2.2. Пространство состояний

Введём дискретное множество скрытых состояний

$$S = \{S_0, S_1, S_2\}$$

где S_0 соответствует нормальному функционированию, S_1 описывает деградацию, а S_2 соответствует предаварийному состоянию. Состояние $S(t)$ трактуется как скрытая переменная, отражающая общий режим работы сервера, тогда как система мониторинга непосредственно наблюдает лишь вектор $X(t)$.

В разрабатываемой системе переход от наблюдаемых метрик к вероятностной оценке состояния формализуется через вектор признаков состояния

$$\varphi(t) = [r_{agg}(t), r_{top1}(t), r_{top3}(t), r_{temp}(t), r_{wear}(t), r_{lat}(t), p_{Markov, S_2}(t)]^T$$

Для каждого состояния S_k , $k \in \{0, 1, 2\}$, вычисляется логит

$$z_k(t) = b_k + \sum_j w_{kj} g_k(\varphi_j(t))$$

Итоговые вероятности состояний вычисляются посредством softmax-нормировки:

$$P(S_k | X(t)) = \frac{\exp(z_k(t)/\tau)}{\sum_l \exp(z_l(t)/\tau)}$$

В результате система получает нормированное распределение вероятностей по состояниям S_0 , S_1 и S_2 , которое далее используется в модулях прогноза риска и поддержки принятия решений.

3. Текст для главы 3

3.1. Прогнозирование временных рядов

Для предупреждения отказов серверного оборудования система должна не только фиксировать текущее состояние, но и оценивать будущую динамику телеметрических показателей на заданных горизонтах прогноза. В разрабатываемой системе используется гибридный прогнозный контур, объединяющий статистическую модель SARIMA и нейросетевую модель LSTM.

Перед построением прогноза временные ряды проходят процедуру STL-декомпозиции, позволяющую отделить трендовую, сезонную и шумовую составляющие:

$$x_i(t) = T_i(t) + S_i(t) + \varepsilon_i(t)$$

В прогнозный модуль передаётся очищенный ряд

$$\tilde{x}_i(t) = T_i(t) + \varepsilon_i(t)$$

что позволяет снизить влияние регулярных суточных и недельных циклов при прогнозировании рисков деградации.

3.2. Статистический прогноз SARIMA

Для статистического прогноза используется модель класса SARIMA, задаваемая уравнением Бокса–Дженкинса:

$$\Phi_p(B) \Phi_p(B^S) (1 - B)^d (1 - B^S)^D x_t = \Theta_q(B) \Theta_q(B^S) \varepsilon_t$$

Практически это означает, что текущее значение метрики оценивается на основе собственных прошлых значений, сезонных повторов и накопленных ошибок прогноза. В реализации используется ограниченный набор устойчивых спецификаций SARIMA, выбираемых из нескольких кандидатов, что обеспечивает баланс между вычислительной стоимостью и устойчивостью работы baseline-модуля.

3.3. Нейросетевой прогноз LSTM

Для описания нелинейной динамики используется рекуррентная нейронная сеть с ячейками LSTM. В общем виде обновление можно записать как

$$(h_t, c_t) = LSTM(x_t, h_{t-1}, c_{t-1})$$

$$\hat{y}_{t+1} = W_o h_t + b_o$$

В текущей реализации нейросетевой модуль работает в виде набора отдельных одномерных LSTM-моделей: для каждой метрики строится собственная модель временного ряда, обучаемая на удалённом вычислительном узле. Такой подход выбран как инженерно устойчивый и менее требовательный к объёму размеченных данных, чем полноценная многомерная LSTM-модель по объединённому вектору аппаратных признаков.

3.4. Интеграция прогнозов SARIMA и LSTM

Итоговый прогноз строится как метрико-специфическая смесь двух моделей:

$$\hat{y}_m(t+h) = \alpha_m \hat{y}_m^{SARIMA}(t+h) + (1 - \alpha_m) \hat{y}_m^{LSTM}(t+h)$$

Для оценки неопределённости итогового прогноза используется интервал, учитывающий как собственную неопределённость каждой модели, так и степень их расхождения:

$$u_m(h) = \sqrt{(\alpha_m u_m^{SARIMA}(h))^2 + ((1 - \alpha_m) u_m^{LSTM}(h))^2 + (\beta |\hat{y}_m^{SARIMA} - \hat{y}_m^{LSTM}|)^2}$$

Итоговый интервал ансамбля задаётся соотношениями:

$$p10 = \hat{y}_m(t+h) - u_m(h), \quad p50 = \hat{y}_m(t+h), \quad p90 = \hat{y}_m(t+h) + u_m(h)$$

4. Текст для главы 4

4.1. Оценка риска отказа

Модуль оценки риска отказа строится как двухконтурная модель. Для изнашиваемых компонентов используется модель Вейбулла, тогда как для остальных эксплуатационных метрик применяется риск приближения к критическому порогу. После этого агрегированный риск по метрикам объединяется с вероятностью перехода в предаварийное состояние, вычисленной по марковской модели.

Для накопителей и других каналов, непосредственно связанных с физическим износом, используется функция надёжности Вейбулла

$$S(t) = \exp\left(-\left(\frac{t}{\eta}\right)^\beta\right)$$

где $\eta > 0$ — параметр масштаба, $\beta > 0$ — параметр формы, а t — текущий уровень износа, выраженный в процентах ресурса, TBW либо другой нормированной характеристике.

Поскольку задача системы состоит не в оценке абстрактной пожизненной надёжности, а в прогнозе риска на конкретном горизонте h , в практической реализации используется условная вероятность отказа на горизонте прогноза:

$$P_{fail}(h | t_0) = 1 - \frac{S(t_f)}{S(t_0)}$$

Для метрик, не являющихся изнашиваемыми, применяется пороговая модель риска. Пусть θ_m — допустимое значение для метрики m , а $p90_m(h)$ — верхняя граница прогнозного интервала на горизонте h . Тогда риск деградации задаётся как

$$r_m(h) = \text{clip}\left(\frac{p90_m(h)/\theta_m - 0.75}{0.75}, 0, 1\right)$$

4.2. Агрегирование риска и марковская динамика состояний

По набору отдельных рисков строится агрегированный риск по метрикам. В реализации он определяется как комбинация взвешенного среднего и максимального риска:

$$R_{metric}(h) = 0.55 r_w(h) + 0.45 \max_m r_m(h)$$

Для описания динамики скрытых состояний используется марковская модель с матрицей переходов

$$P = \begin{pmatrix} p_{00} & p_{01} & 0 \\ 0 & p_{11} & p_{12} \\ 0 & 0 & 1 \end{pmatrix}$$

Вектор текущих вероятностей состояния обозначим как

$$\pi_t = [P(S_0 | X_t), P(S_1 | X_t), P(S_2 | X_t)]$$

Тогда прогноз распределения состояний через k шагов вычисляется по формуле

$$\pi_{t+k} = \pi_t P^k$$

Итоговый риск отказа вычисляется как смесь марковского риска и агрегированного риска метрик:

$$R_{overall}(h) = w_M \pi_{t+k}(S_2) + (1 - w_M) R_{metric}(h)$$

Надёжность системы на выбранном горизонте определяется как

$$Rel(h) = 1 - R_{overall}(h)$$

5. Текст для главы 6

5.1. Интегральная математическая модель системы поддержки принятия решений

Рассмотренные ранее модули прогнозирования, оценки состояния, расчёта риска и выбора действий образуют единый вычислительный контур. Поэтому разрабатываемую систему целесообразно описать как композиционный оператор, который преобразует поток телеметрических данных в управленческое решение.

Пусть $X(t)$ — вектор текущих наблюдаемых метрик, а Θ — совокупность параметров системы, включающая параметры STL-декомпозиции, прогнозных моделей, модели состояний, марковской динамики и матрицы потерь. Тогда итоговое рекомендуемое действие a^* определяется как

$$a^* = F(X(t), \Theta)$$

С учётом фактической архитектуры системы оператор F целесообразно представить в виде следующей композиции:

$$F = AHP \circ Decision \circ RiskFusion \circ Markov \circ StateInference \circ Ensemble \circ \{SAR\}$$

Эта запись читается справа налево. Сначала сырые телеметрические ряды проходят предварительную обработку и очистку. Далее для каждого канала строятся статистический и нейросетевой прогнозы, после чего формируется ансамблевый прогноз с оценкой неопределённости. На основании прогнозных точек вычисляется вероятностная оценка скрытого состояния сервера S_0 , S_1 , S_2 . Затем по вероятностям текущего состояния строится марковский прогноз динамики состояний на выбранном горизонте. Параллельно вычисляется риск по метрикам: через модель Вейбулла для изнашиваемых компонентов и через пороговую модель для остальных показателей. Эти компоненты объединяются в интегральный риск отказа. Наконец, модуль принятия решений оценивает ожидаемые потери альтернатив и, при необходимости, корректирует выбор с помощью многокритериальной АНР-свёртки.

Формально контур оценки риска можно представить как

$$R(h) = G_{risk}(\pi_t, P, \hat{X}_{t+h})$$

где $\hat{X}_{t+h}$ — ансамблевый прогноз метрик на горизонте h , π_t — текущая вероятностная оценка состояния, P — матрица переходов марковской модели. Тогда модуль выбора действия реализует задачу

$$a^* = \arg \min_{a_i \in A} E[L(a_i, S)]$$

в байесовской постановке, а в многокритериальном режиме использует глобальную полезность

$$U(a_i) = \sum_k w_k u_k(a_i)$$

Таким образом, интегральная модель объединяет в одной вычислительной схеме методы обработки временных рядов, вероятностного прогноза, оценки надёжности и теории принятия решений. Научная ценность предлагаемого подхода состоит в том, что отдельные модули не используются изолированно, а образуют согласованный контур: от наблюдения телеметрии до формализованной рекомендации действия.

6. Разъяснения по ключевым вопросам

6.1. Что означает множество состояний $S = \{S_0, S_1, S_2\}$

Здесь важно не перепутать два разных объекта: множество состояний и вектор признаков. Множество состояний S в текущей модели состоит ровно из трёх элементов:

$$S = \{S_0, S_1, S_2\}$$

Это не означает, что в системе появилось много новых скрытых состояний. Напротив, семантика в интерфейсе остаётся прежней и полностью сохраняется:

- S_0 — сервер работает в нормальном режиме; - S_1 — сервер демонстрирует признаки деградации; - S_2 — сервер находится в предаварийном состоянии.

Изменился не набор состояний, а способ их вычисления. Ранее переход к S_0 / S_1 / S_2 был эвристическим. Теперь эти три состояния оцениваются вероятностно на основе набора признаков:

$$\pi_t = (P(S_0 | X_t), P(S_1 | X_t), P(S_2 | X_t))$$

Следовательно, в UI по-прежнему отображаются те же самые три состояния, но теперь за ними стоит не набор жёстких правил, а формальная модель вероятностей.

Количество метрик при этом влияет не на число состояний, а на размер информационного описания $X(t)$ и на состав компонент риска. Если в систему добавляются новые метрики, множество S не расширяется автоматически. Расширяется вектор наблюдений и, при необходимости, вектор признаков состояния $\varphi(t)$. Иными словами: состояний всё так же три, а информации для их оценки становится больше.

6.2. Почему в baseline используется очищенный ряд $\tilde{x}(t) = T(t) + \epsilon(t)$

В текущей baseline-реализации STL применяется для того, чтобы отделить регулярную сезонную волну от более важной для диагностики части сигнала:

$$x_i(t) = T_i(t) + S_i(t) + \epsilon_i(t), \quad \tilde{x}_i(t) = T_i(t) + \epsilon_i(t)$$

Практический смысл такого шага состоит в следующем. Если метрика имеет устойчивый суточный или недельный цикл, то модель может начать «обучаться расписанию», а не деградации. Для диагностической задачи это рискованно: система будет слишком хорошо повторять нормальную рутину, но хуже ловить медленное ухудшение состояния.

Плюсы текущего подхода:

- статистическая модель меньше подстраивается под чисто календарный ритм; - тренд деградации виден лучше; - прогноз baseline становится устойчивее на короткой истории.

Однако у такого решения есть и важное ограничение: в текущем коде сезонная компонента не добавляется обратно в итоговый прогноз. Это означает, что сезонность не игнорируется полностью, но используется как этап очистки, а не как явная часть конечного прогноза.

Если в диссертации требуется сделать акцент именно на том, что система учитывает сезонность в итоговом прогнозе, то корректнее перейти к одному из двух вариантов:

1. либо выполнять STL-декомпозицию, прогнозировать очищенный ряд и затем добавлять обратно прогноз сезонной компоненты; 2. либо обучать SARIMA непосредственно на исходном ряду с сезонными параметрами и сравнивать такой вариант с декомпозицией.

Следовательно, текущий baseline научно корректен как упрощённая диагностическая схема, но для более сильной формулировки о «полноценном учёте сезонности» его желательно доработать.

6.3. Что означает «не полный перебор порядков» в SARIMA

SARIMA задаётся набором порядков

$$(p, d, q) \times (P, D, Q, s)$$

где p, d, q описывают обычную часть модели, а P, D, Q, s — сезонную. Полный автоматический перебор означает, что система перебирает много комбинаций этих параметров, например:

$$p, q, P, Q \in \{0, 1, 2, 3\}, \quad d, D \in \{0, 1, 2\}$$

после чего по каждому кандидату выполняется подгонка модели и выбирается лучший вариант по критерию качества, например AIC, BIC или ошибке на валидации.

В текущем baseline этого нет. Сейчас система пробует только ограниченный набор устойчивых конфигураций. Причина практическая: полный search для каждой метрики, каждого устройства и каждого запуска резко увеличивает время расчёта и может давать нестабильные или плохо сходящиеся модели.

Следовательно, текущее решение — это не ошибка, а инженерный компромисс. Но если цель именно диссертационная полнота, то следующий шаг очевиден: реализовать автоматический подбор SARIMA-порядков с ограниченным, но уже расширенным grid-search и сохранением выбранной спецификации в результатах запуска.

6.4. Почему текущая LSTM одномерная и когда нужна многомерная LSTM

Сейчас в системе реализована одномерная схема: для каждой метрики строится отдельная LSTM-модель. Это означает, что, например, температура RAID, память и CPU прогнозируются раздельно.

Такой подход удобен как baseline для нейросети, потому что он:

- проще в обучении; - устойчивее на малом количестве данных; - легче отлаживается; - даёт понятное сравнение «одна метрика — один прогноз».

Но если научная цель состоит в том, чтобы показать преимущество нейросети именно за счёт учёта взаимосвязей между каналами телеметрии, то для диссертации действительно лучше переходить к многомерной LSTM. В этом случае сеть получает на вход окно из нескольких метрик одновременно, например:

$$X_t = [x_t^{cpu}, x_t^{mem}, x_t^{temp}, x_t^{net}, x_t^{raid}]$$

Тогда модель может использовать межметрические зависимости: рост температуры вместе с ростом нагрузки, рост памяти вместе с деградацией отклика и т.д. Именно это и является главным научным аргументом в пользу нейросети по сравнению с SARIMA.

Следовательно, здесь вывод такой:

- если нужна простая и устойчивая рабочая реализация — одномерные LSTM допустимы; - если нужна диссертационно сильная формулировка о преимуществе нейросетевого подхода, правильнее реализовать многомерную LSTM.

Для темы работы более убедительным выглядит второй вариант.

6.5. Что означает неопределённость ансамбля простым языком

Итоговый ансамблевый прогноз даёт не только одно число, но и коридор возможных значений:

$$p10, \quad p50, \quad p90$$

Простой смысл здесь такой:

- **p50** — центральный прогноз, наиболее типичный ожидаемый уровень; - **p10** — более осторожная нижняя граница; - **p90** — более осторожная верхняя граница.

Ширина этого коридора задаётся величиной $u_m(h)$:

$$u_m(h) = \sqrt{(\alpha_m u_m^{SARIMA}(h))^2 + ((1 - \alpha_m) u_m^{LSTM}(h))^2 + (\beta |y_m^{SARIMA} - y_m^{LSTM}|)^2}$$

Её можно понимать так:

1. если сама SARIMA неуверенна, её вклад в ширину интервала растёт; 2. если сама LSTM неуверенна, её вклад тоже растёт; 3. если две модели сильно расходятся между собой, коридор дополнительно расширяется.

Именно поэтому такая формула полезна. Если просто усреднить два прогноза и не учитывать их расхождение, система будет выглядеть излишне уверенной даже там, где модели между собой не согласны. Для СППР это плохо: пользователь получит красивое число, но не увидит, насколько оно надёжно.

С практической точки зрения учёт uncertainty улучшает систему следующим образом:

- риск вычисляется не только по «лучшей точке», но и по неблагоприятной верхней границе p_{90} ;
- рекомендации становятся более осторожными там, где прогноз нестабилен; - система честнее показывает неопределённость, что особенно важно для диссертационной темы «риск и неопределённость».

6.6. Почему для Вейбулла важнее горизонтная вероятность отказа, чем сама функция выживания

Функция Вейбулла

$$S(t) = \exp\left(-\left(\frac{t}{\eta}\right)^\beta\right)$$

показывает вероятность того, что объект в принципе «доживёт» до момента t . Но для практической СППР этого недостаточно. Система должна отвечать не на абстрактный вопрос «насколько надёжен диск вообще», а на конкретный вопрос:

«Какова вероятность отказа в ближайшие 24 часа, 7 дней или 30 дней, если диск уже дожил до текущего момента?»

Именно это и даёт горизонтная вероятность отказа:

$$P_{fail}(h | t_0) = 1 - \frac{S(t_f)}{S(t_0)}$$

Здесь t_0 — текущее состояние износа, а t_f — ожидаемое состояние на горизонте прогноза. Такая формула условная: она считает риск именно на ближайшем интервале времени, а не «с нуля от рождения компонента».

Для принятия решений это принципиально лучше, потому что:

- именно горизонтный риск связан с конкретным окном обслуживания; - он позволяет сравнивать 24h, 7d и 30d между собой; - он даёт величину, которую можно напрямую использовать в СППР и в экономике решений.

Следовательно, функция выживания нужна как фундамент модели, а горизонтная вероятность отказа — как практический инструмент принятия решений.

6.7. Почему для неизнашиваемых метрик вводится отдельный пороговый риск

Не все метрики сервера описываются моделью износа. Для CPU, памяти, температуры и сетевых показателей логика другая: компонент может быть исправен физически, но система всё равно входит в опасный режим из-за перегрева, перегрузки или ухудшения отклика.

Поэтому для таких метрик используется не Вейбулл, а риск приближения к критическому режиму:

$$r_m(h) = \text{clip}\left(\frac{p_{90m}(h)/\theta_m - 0.75}{0.75}, 0, 1\right)$$

Смысл формулы следующий:

- θ_m — допустимый порог; - $p_{90_m}(h)$ — «осторожный верхний прогноз» на горизонте h ; - если $p_{90_m}(h)$ заметно ниже порога, риск близок к нулю; - если прогноз подходит к порогу, риск плавно растёт; - если прогноз превышает порог, риск стремится к единице.

Почему здесь берётся именно p_{90} , а не p_{50} ? Потому что СППР по риску должна смотреть на неблагоприятный, но ещё реалистичный сценарий. Это делает систему более пригодной для предупредительной диагностики.

Такой подход можно интерпретировать как модель эксплуатационной деградации. Компонент может ещё не быть сломанным, но режим его работы уже становится опасным. Именно это и требуется для серверного мониторинга: заранее увидеть переход к плохому режиму, а не ждать фактического отказа.

Если метрика двусторонняя, например опасно и слишком высокое, и слишком низкое значение, то модель можно расширить до двустороннего риска. Однако для большинства текущих серверных метрик основная опасность действительно односторонняя: перегрев, рост задержек, рост загрузки, рост использования памяти.

6.8. Зачем нужен общий риск отказа и почему он строится как смесь

Один источник информации не даёт полной картины. Агрегированный риск по метрикам хорошо показывает непосредственную опасность «здесь и сейчас», но не отражает инерцию деградации. Марковская модель, наоборот, хорошо описывает накопленную динамику переходов между S_0 , S_1 , S_2 , но может быть слишком грубой, если внезапно вспыхнула одна конкретная метрика.

Поэтому вводится общий риск:

$$R_{overall}(h) = w_M \pi_{t+k}(S_2) + (1 - w_M) R_{metric}(h)$$

Смысл этой смеси таков:

- $R_{metric}(h)$ отражает силу текущих и прогнозных опасных метрик; - $\pi_{t+k}(S_2)$ отражает вероятность перехода системы в предаварийный режим по динамике состояний; - параметр w_M задаёт баланс между «моментной опасностью» и «накопленной тенденцией деградации».

Это улучшает модель сразу в нескольких отношениях.

Во-первых, система становится устойчивее к ложным одиночным всплескам. Если одна метрика кратковременно изменилась, но история состояний стабильна, общий риск не взлетит слишком резко.

Во-вторых, система не пропускает медленную деградацию. Если отдельные метрики ещё не выглядят критично, но сервер уже давно движется по траектории $S_0 \rightarrow S_1 \rightarrow S_2$, общий риск будет расти раньше.

В-третьих, общий риск удобен для СППР. Именно одну интегральную величину легче связывать с ожидаемыми потерями, классами рекомендаций и порогами принятия решений.

Следовательно, общий риск отказа — это не косметическое усреднение, а способ объединить локальные сигналы метрик и глобальную динамику деградации в одну управленчески полезную оценку.